

Distributed Systems - 19_11

Our specification of the total order broadcast will have the following properties:

- Validity
- Integrity: we cannot invent messages from thin air, and we cannot duplicate messages
- Agreement: at least the correct processes see the same set of messages
- Order: we want to same ordering at least on the correct processes

Reminder that for agreement we have two flavors:

- Uniform: if a process delivers a message, then all the correct processes will deliver that message. The process could either be correct or faulty, and the messages delivered by a faulty will be a subset of the delivered messages from a correct
- Non Uniform: If a correct process delivers a message, then all correct processes will deliver that message. This means that it's not necessarily true the statement of the subset described above. A process that is faulty could deliver a message that is not delivered by the correct processes.

Let's see the interface.

Module:
Name: TotalOrderBroadcast, **instance** *tob*.
Events:
Request: $\langle tob, Broadcast \mid m \rangle$: Broadcasts a message m to all processes.
Indication: $\langle tob, Deliver \mid p, m \rangle$: Delivers a message m broadcast by process p .
Properties:
TOB1: Validity: If a correct process p broadcasts a message m , then p eventually delivers m .
TOB2: No duplication: No message is delivered more than once.
TOB3: No creation: If a process delivers a message m with sender s , then m was previously broadcast by process s .
TOB4: Agreement: If a message m is delivered by some correct process, then m is eventually delivered by every correct process.
TOB5: Total order: Let m_1 and m_2 be any two messages and suppose p and q are any two correct processes that deliver m_1 and m_2 . If p delivers m_1 before m_2 , then q delivers m_1 before m_2 .

For the last property, all pairs of messages for two processes have to have the same order of the messages.

We can't have something like this:

m_1	m_2	m_4	m_3		p_1
m_1	m_2	m_3	m_4		p_2

To implement this algorithm, we'll use:

- the Reliable Broadcast
- the Consensus block, which could be either the hierarchical one, or Paxos, and so on. This block takes a propose from a certain process, and gives a decision to the total order broadcast block.

Let's take a look at the algorithm:

```

Implements:
  TotalOrderBroadcast, instance tob.

Uses:
  ReliableBroadcast, instance rb;
  Consensus (multiple instances).

upon event  $\langle tob, Init \rangle$  do
  unordered :=  $\emptyset$ ;
  delivered :=  $\emptyset$ ;
  round := 1;
  wait := FALSE;

  upon event  $\langle c.r, Decide \mid decided \rangle$  such that  $r = round$  do
    forall  $(s, m) \in sort(decided)$  do // by the order in
      trigger  $\langle tob, Deliver \mid s, m \rangle$ ; // in the resulting sorted list
    delivered := delivered  $\cup$  decided;
    unordered := unordered  $\setminus$  decided;
    round := round + 1;
    wait := FALSE;

  upon event  $\langle tob, Broadcast \mid m \rangle$  do
    trigger  $\langle rb, Broadcast \mid m \rangle$ ;

  upon event  $\langle rb, Deliver \mid p, m \rangle$  do
    if  $m \notin delivered$  then
      unordered := unordered  $\cup$   $\{(p, m)\}$ ;

```

In the unordered I put the messages that are pending, as we do not know what will be the order of the messages. In delivered we put the messages that have been ordered.

The round is the round number, the algorithm works with logical rounds (it's not a physical round). wait is a value that we use to remember that there is an instance of consensus running, remembering that we don't have to start another instance when that instance is running.

When a process wants to broadcast a message, we trigger the broadcast of the reliable broadcast.

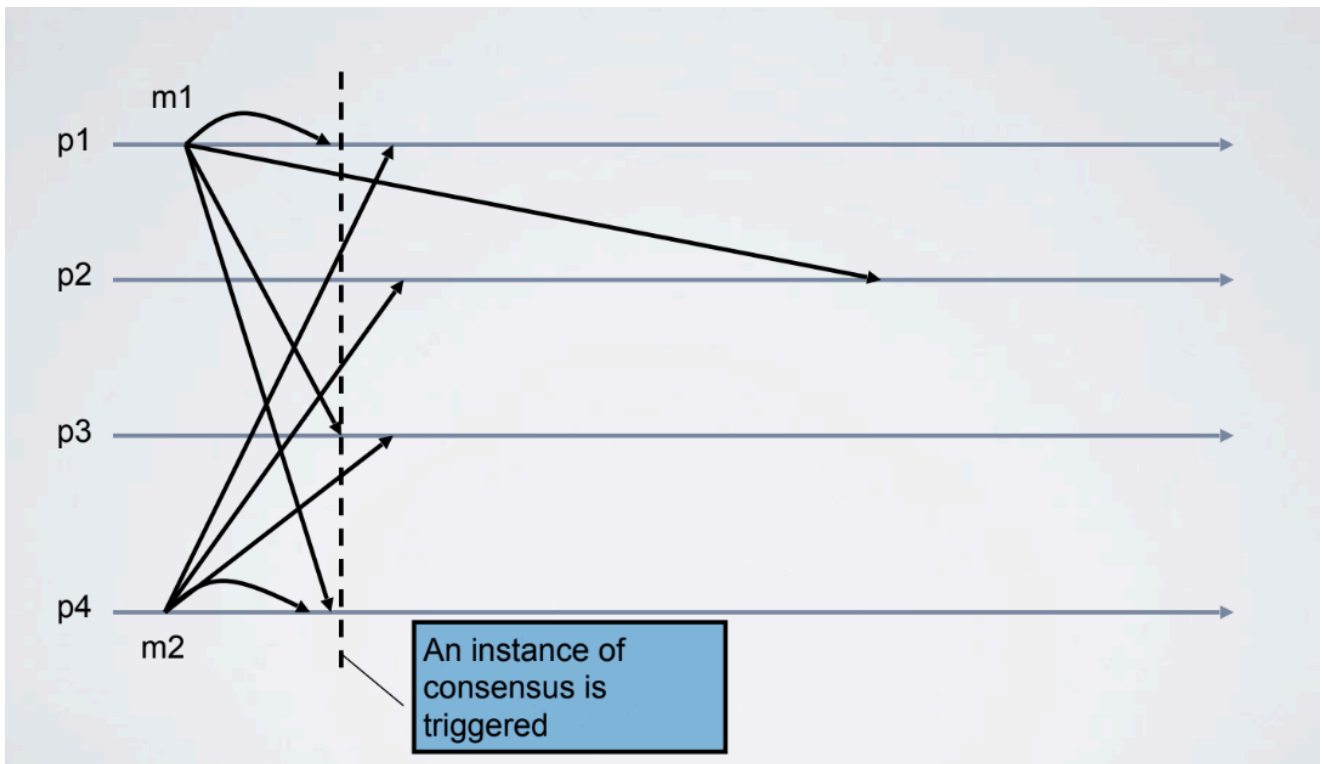
When we deliver a message, I check if it's new, and if yes we put the message received in the unordered set.

When the process has a message in unordered, and there is not an instance of consensus running, we set wait to True and we initialize a new instance of consensus for that round, proposing the set unordered.

When the consensus terminates, the messages in the unordered set will be decided in all processes, with the messages decided being the same for everyone. We trigger the handler, unpack the decision, and start delivering the messages. We take the decided set, and sort it. In the unordered set we'll remove the messages delivered, increase the round and set wait to false.

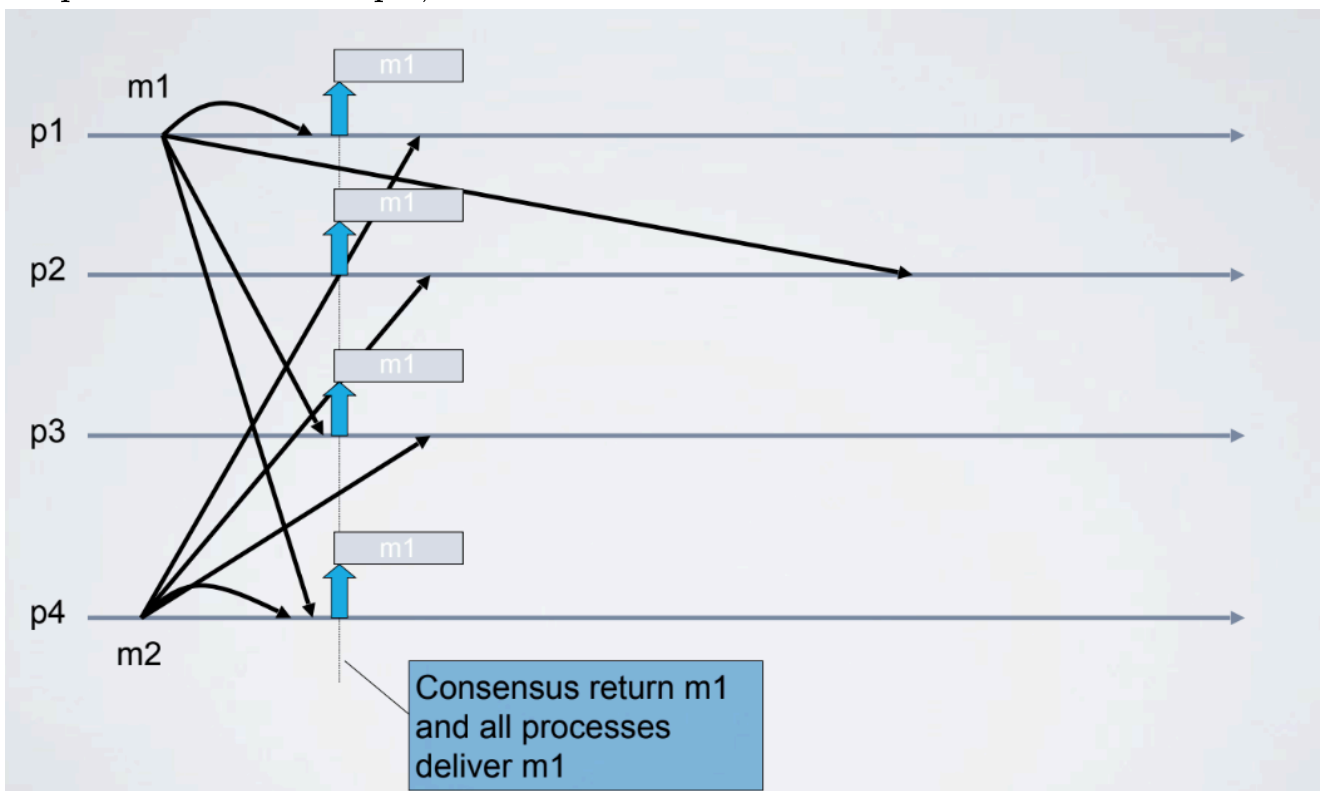
Let's take a look at an execution.

Note that, in order to identify the instance of consensus, we'll have to send the message alongside a number associated with the message.

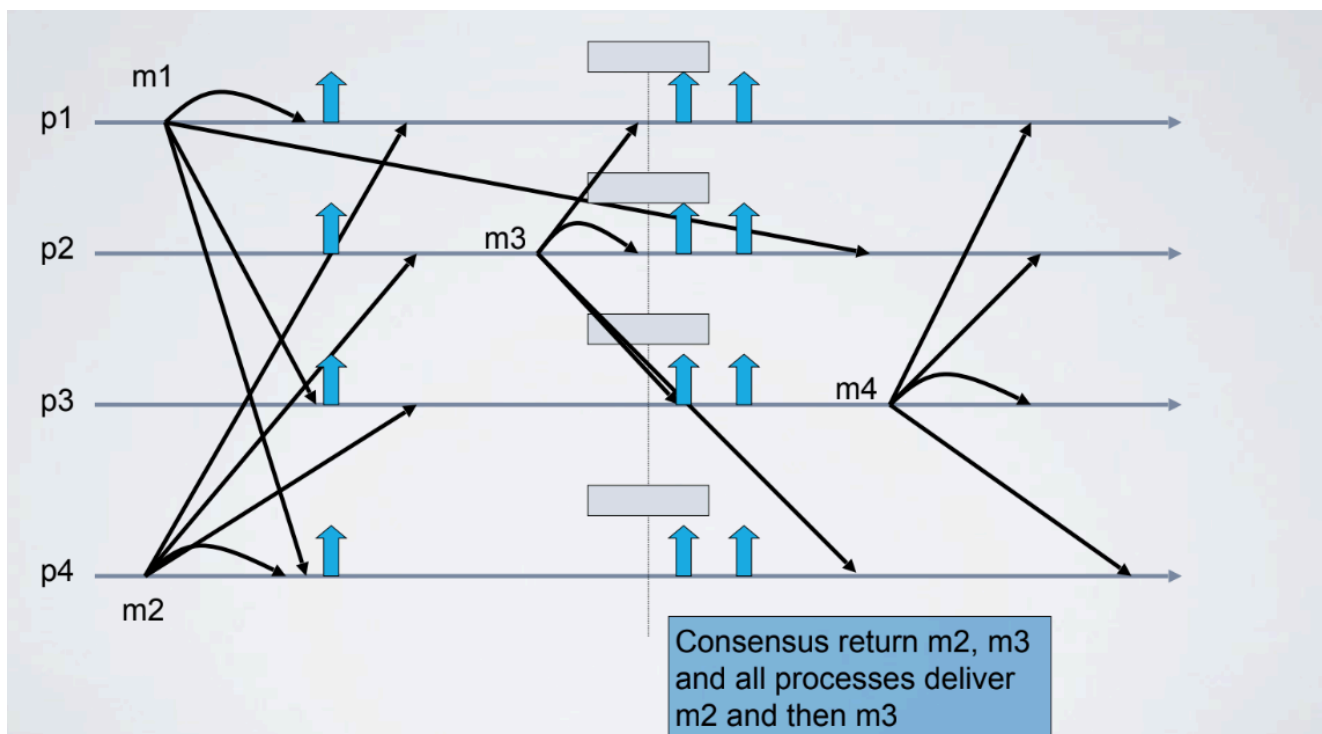
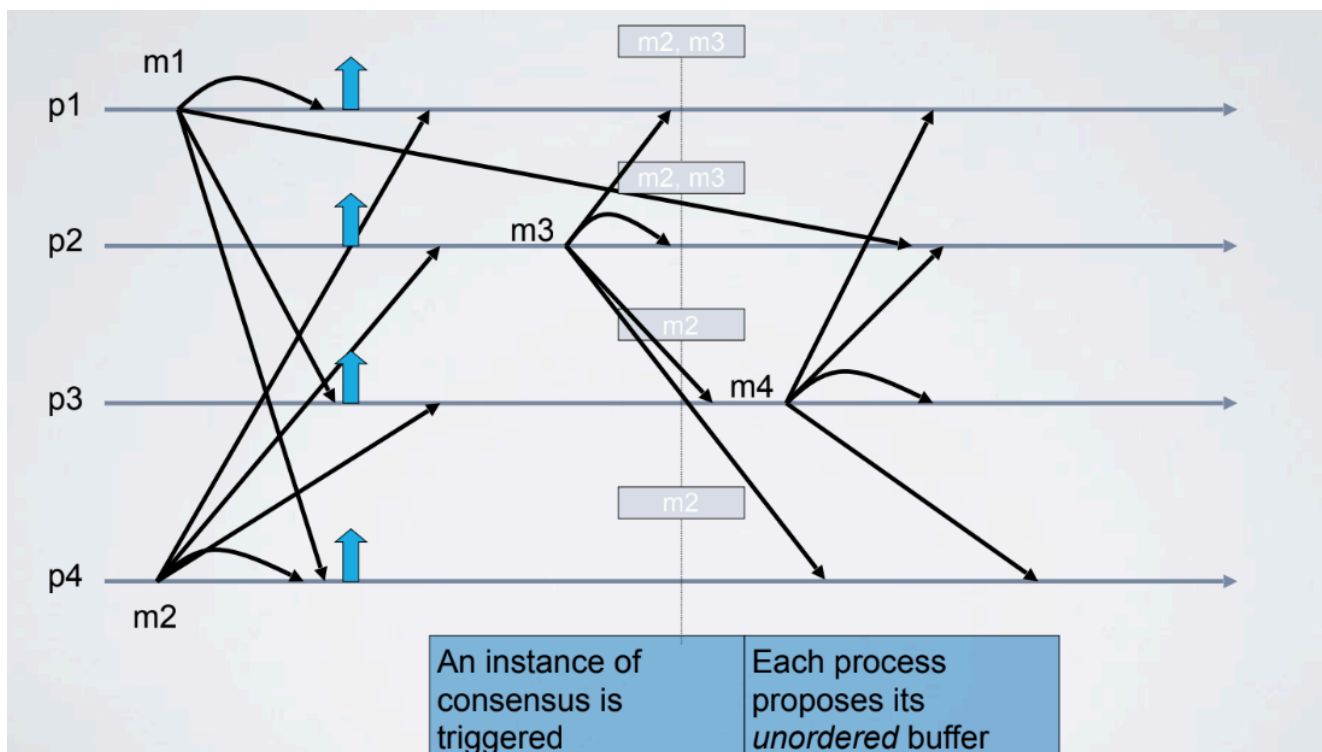


p_4 will propose, for example, $U : \{(p_1, m_1), (p_4, m_2)\}$, while p_3 will propose $U : \{(p_1, m_1)\}$.

The consensus will decide at some instance, and it could happen at different times for processes. For example, it will decide m_1 .



Once m_1 is decided, for p_2 we'll have that $U : \{(p_4, m_2)\}$, for p_3 we'll have $U : \emptyset$ and for p_4 we'll have $U : \{(p_4, m_2)\}$.



When m_1 will finally arrive at p_2 , we'll discard it, as it is already in the delivered set.

Let's take a look at the proof.

The no creation: If we deliver a message, then it was sent by someone. Where are the points where a message can be created? One is in the Deliver handler, and is it possible that we create a message there? No, because of the no creation property of the Reliable Broadcast.

No duplication: Suppose by contradiction that we deliver a message two times. It could happen in two ways: the first way in which it could happen is that the instance of round 10 of consensus triggers, and in that instance we deliver the

same messages twice. Is it possible? No, as the set decided is indeed a set, so it contains unique elements. So, we would need two different rounds in which we deliver the same messages. Then the round number of one is smaller than the other, and so the first round happens before the other. When we decide at round 5 (for example), we put message m_1 in the delivered set, so it's not possible that this happens as for the consensus. *check this better* Also, along side putting the message in the delivered, we also remove it from unordered, so a message cannot appear anymore (also by the fact that now the message is in delivered for everyone, and so even if the message is delivered with a delay, it won't be put in unordered).

Agreement:

Agreement: Suppose a correct process p_0 delivers m at round r . Then m is in decided of the consensus of round r .

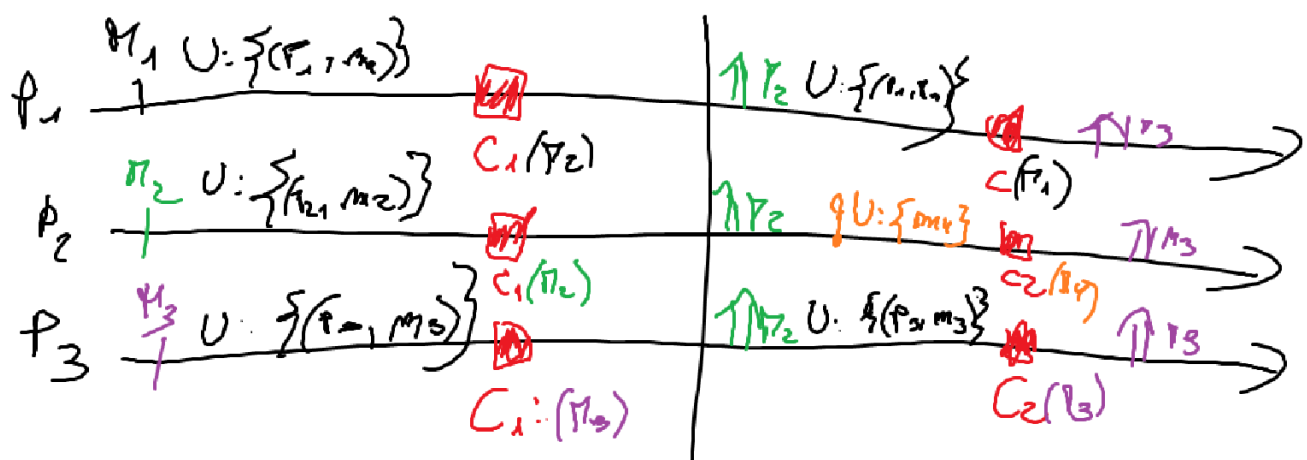
If m is not in the decided of the consensus of round r of a correct process p_1 , which **property of consensus are you violating?**

So m has to be in the decided of any correct process at round r . -> agreement.

It remains to show the total order. We show it by contradiction. Let's take that p_1 first delivers m_1 and then m_2 , and p_2 delivers m_2 and then m_1 . Suppose m_1 and m_2 are delivered in the same round, then it means that we deliver them in the for. If there are two processes delivering a message, then it would mean that decided is different for two processes in a certain round r , thus violating consensus properties.

Here we are using a reliable broadcast to send the message to others, what happens if we modify the algorithm, such that when we want to broadcast a message, we put it immediately in the unordered set?

The only property that could be problematic is the validity, here is an execution:



Is it mandatory that m_1 eventually wins with this algorithm? Not really, we could

have a case where m_1 is given to the consensus, but it never wins the consensus. Thus, we are violating validity.

Why is it different from the normal case? If we use rb then eventually every correct process will have m_1 , and so even if p_1 never wins then another correct process having m_1 will win, and thus m_1 will be delivered.

This total order broadcast that we saw is not uniform, in fact if we have p_1 correct and p_2 faulty, it's possible that p_2 decides different things from p_1 . How do we fix this problem? We want to create a total order broadcast that is uniform, with the uniform agreement property.

We could make the algorithm uniform without changing a single line. To violate the ordering, two processes have to violate the ordering in the for. Let's take p_1 correct and p_2 faulty. If the consensus is not uniform, we might have p_1 decide a value for a round, and p_2 another value for that round. If I use a consensus with uniformity, like the one with ACKs, is it possible? No, as for the fact the consensus is uniform, so if we make a decision for someone we have to make the same decision for everyone else.

So, we have to use uniform consensus to get uniform total order, and not uniform for not uniform total order.

Also, uniform reliable broadcast is not enough to get uniform total order.

Does the algorithm enforce also a FIFO order or causal order? It depends on the sort operation, for example if we have $m_1 = 7$ and $m_2 = 0$ (with m_1 sent before m_2), we have that for the sort first $m_2 = 0$ will be delivered, then $m_1 = 7$, thus breaking FIFO.